CEFET/RJ

TÉCNICAS DE APRENDIZADO DE MÁQUINA

Bacharelado em Sistemas da Informação

Prof. Marco André Abud Kappel

Aula 3 – Pré-processamento de dados

Pré-processamento de dados

- **Introdução**
  - Antes de aplicarmos **algoritmos** de Machine Learning em uma base dados, precisamos fazer um **pré-processamento** dos dados.
  - Em muitas bases de dados reais existem algumas **inconsistências** nos dados.
  - ➢ **Exemplos**:
    - Valores corrompidos.
    - Valores faltantes.
    - Tratamento com escalas
    - Transformação de variáveis categóricas.
    - Separação em base de treinamento e base de teste.

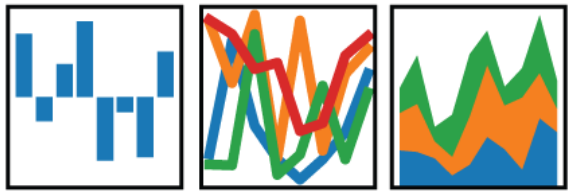
Pré-processamento de dados

- **Introdução**
  - Para fazer uma manipulação eficiente dos dados, vamos utilizar vários recursos do pandas:

    - ❑ Leitura de dados
    - ❑ Visualização de Dataframes
    - ❑ Localizar dados
    - ❑ Remover linhas e colunas
    - ❑ Alterar valores

pandas

$y_i t = \beta' x_{it} + \mu_i + \epsilon_{it}$



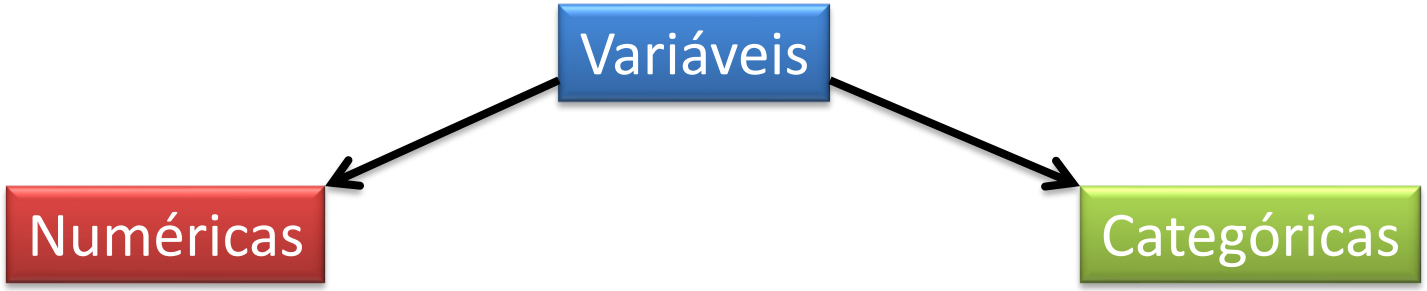
Pré-processamento de dados

- **Tipos de variáveis**
  - Uma base de dados real irá possuir informações de diversos **tipos** diferentes.
  - Dependendo do tipo do dado, pode ser necessário fazer alguma **conversão** antes de executar um algoritmo.
  - Alguns algoritmos aceitam apenas valores **numéricos**, enquanto outros podem ser capazes apenas de lidar com variáveis **categóricas**.
  - É necessário, então, identificar os tipos corretos de cada atributo do problema **antes** de começar a tentar resolvê-lo.

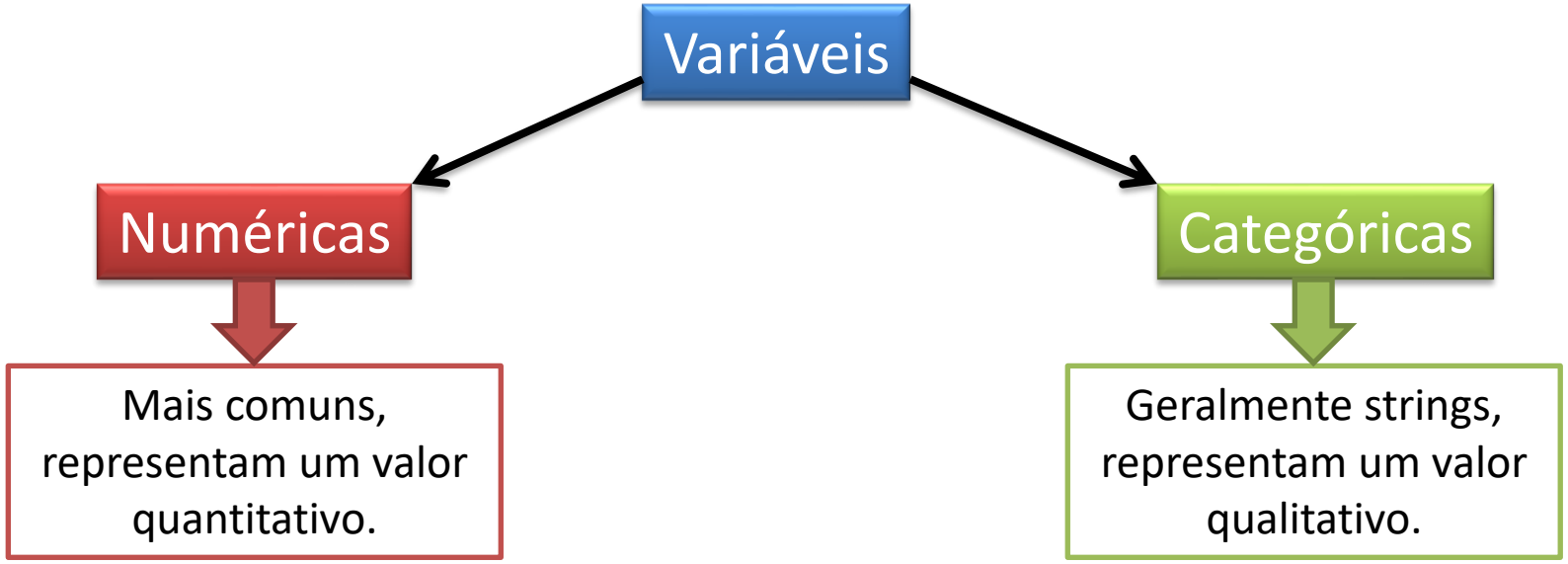
- **Tipos de variáveis**

Variáveis

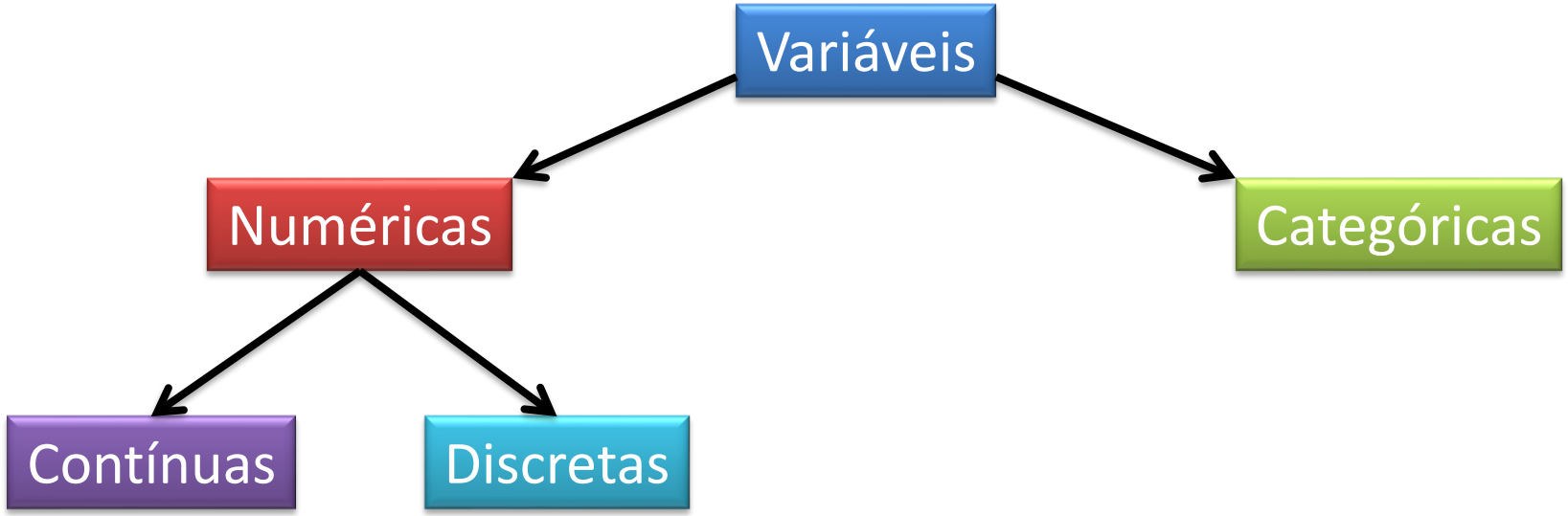
- **Tipos de variáveis**



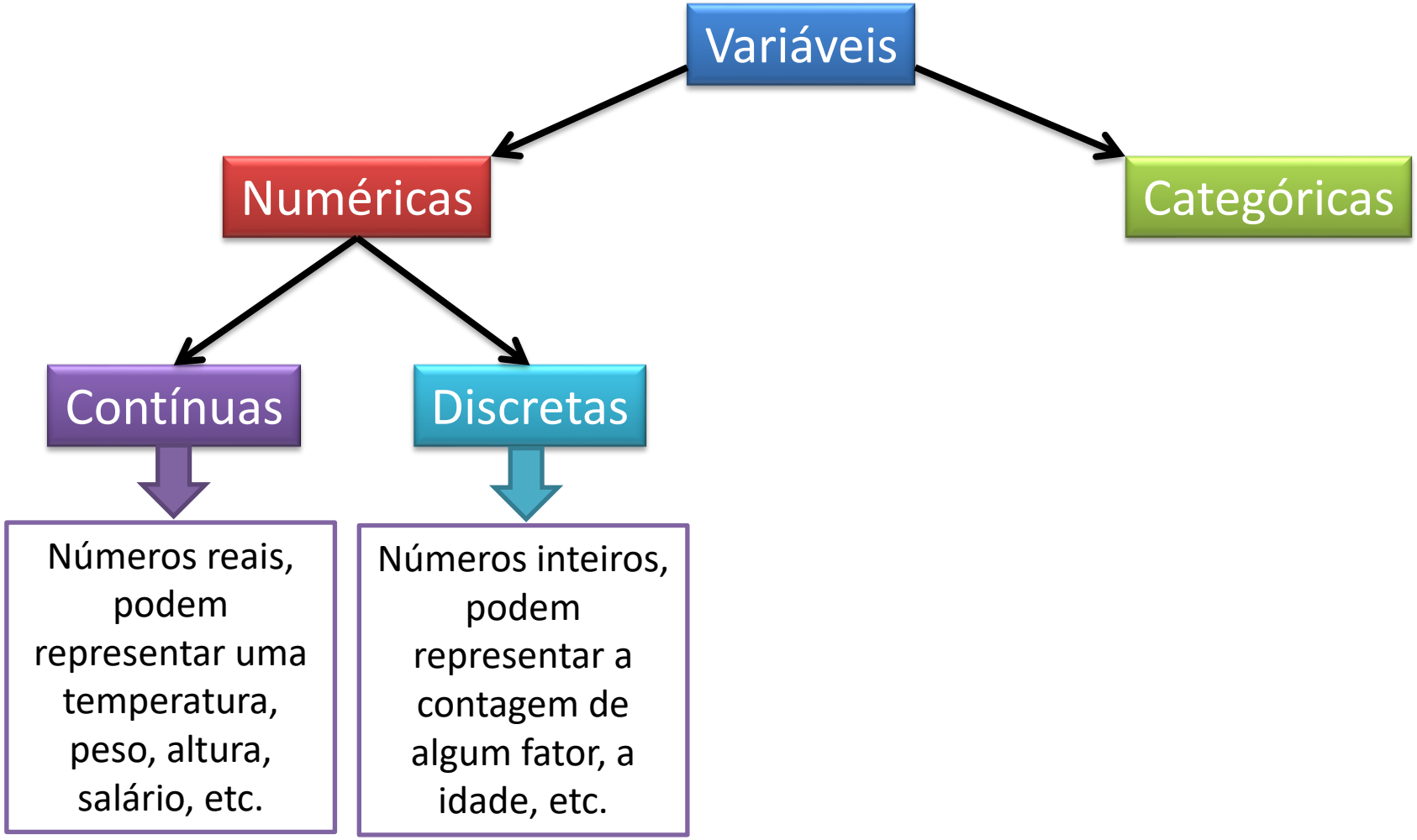
- **Tipos de variáveis**



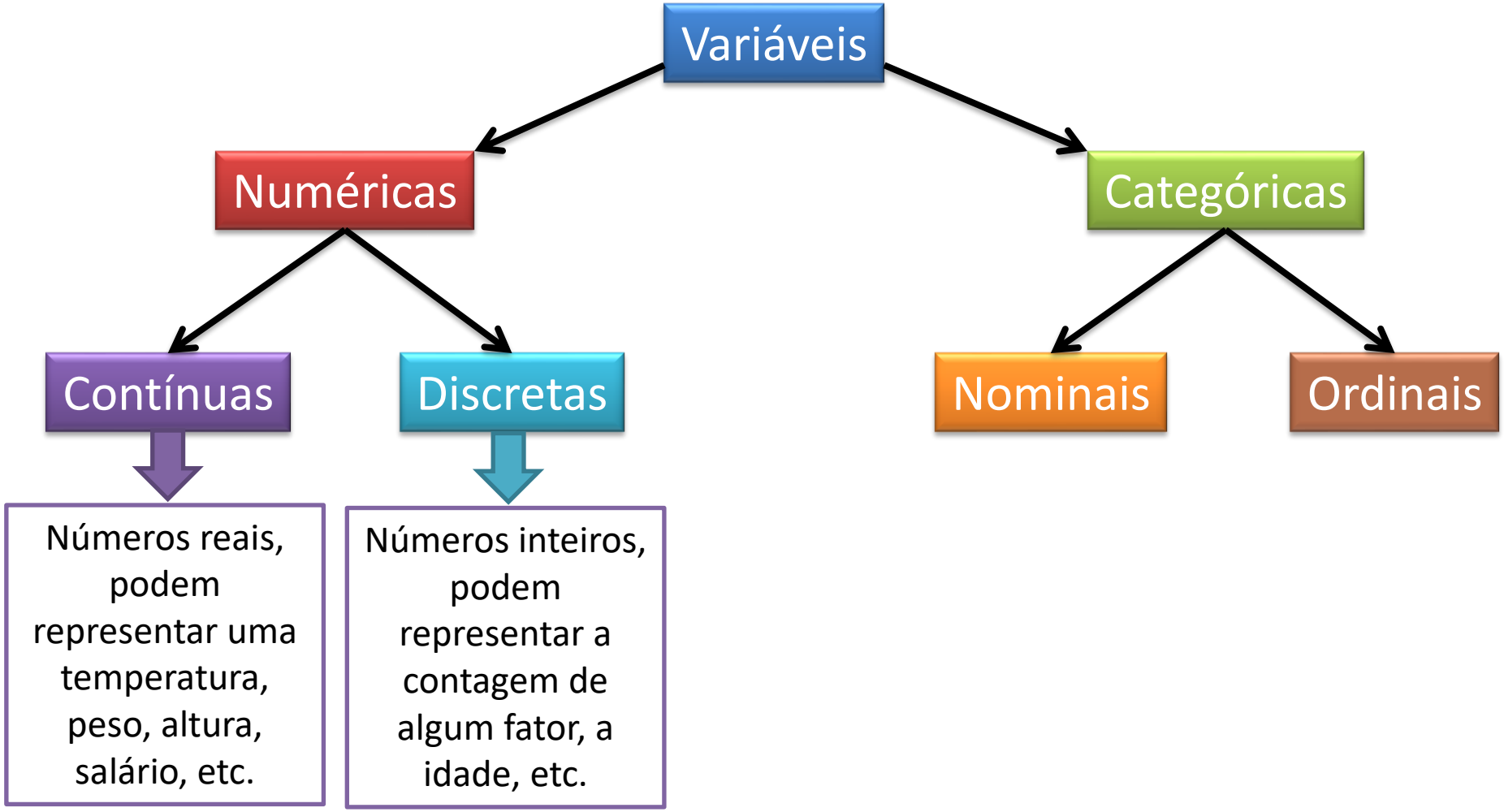
- **Tipos de variáveis**



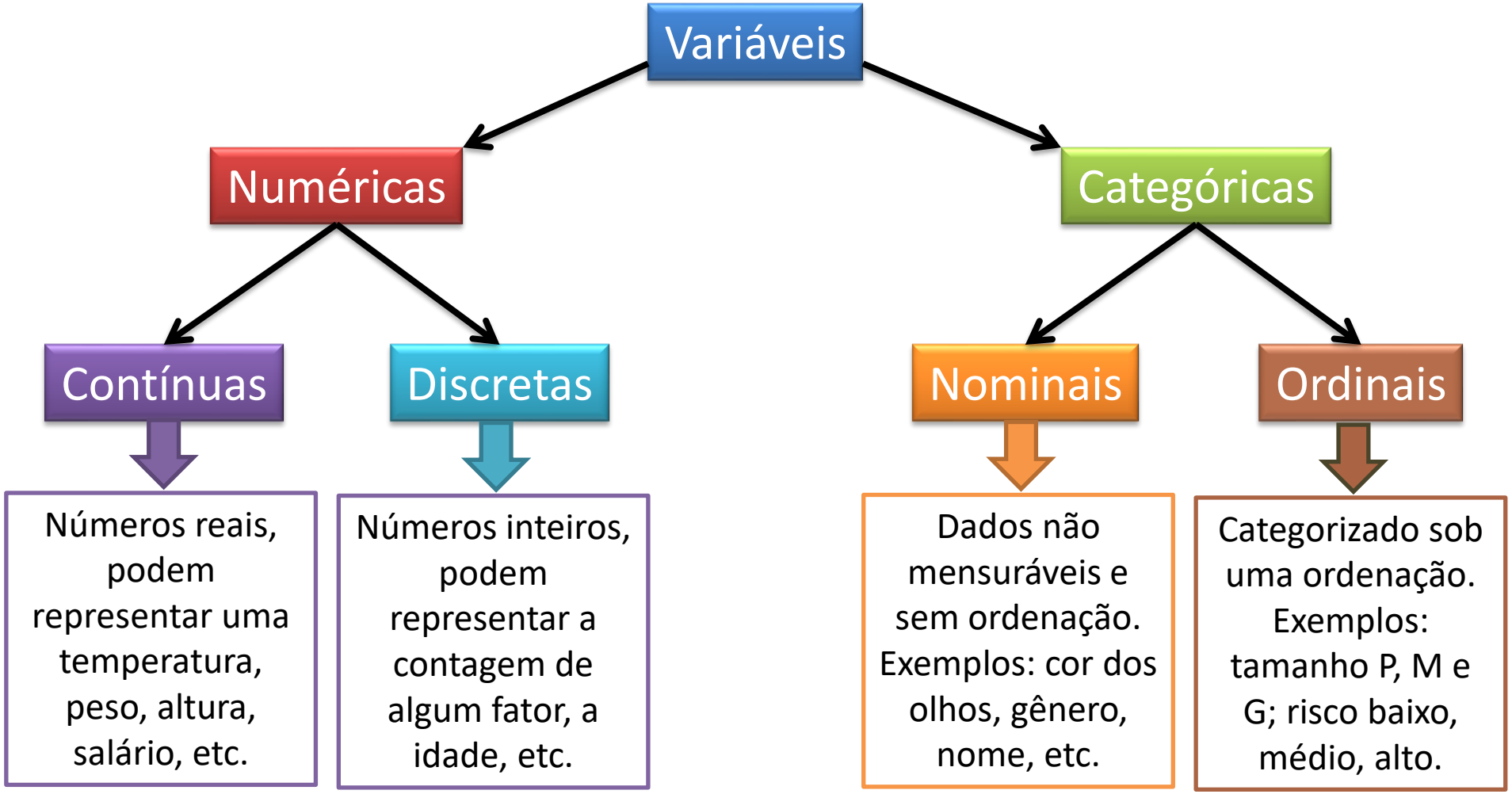
- **Tipos de variáveis**



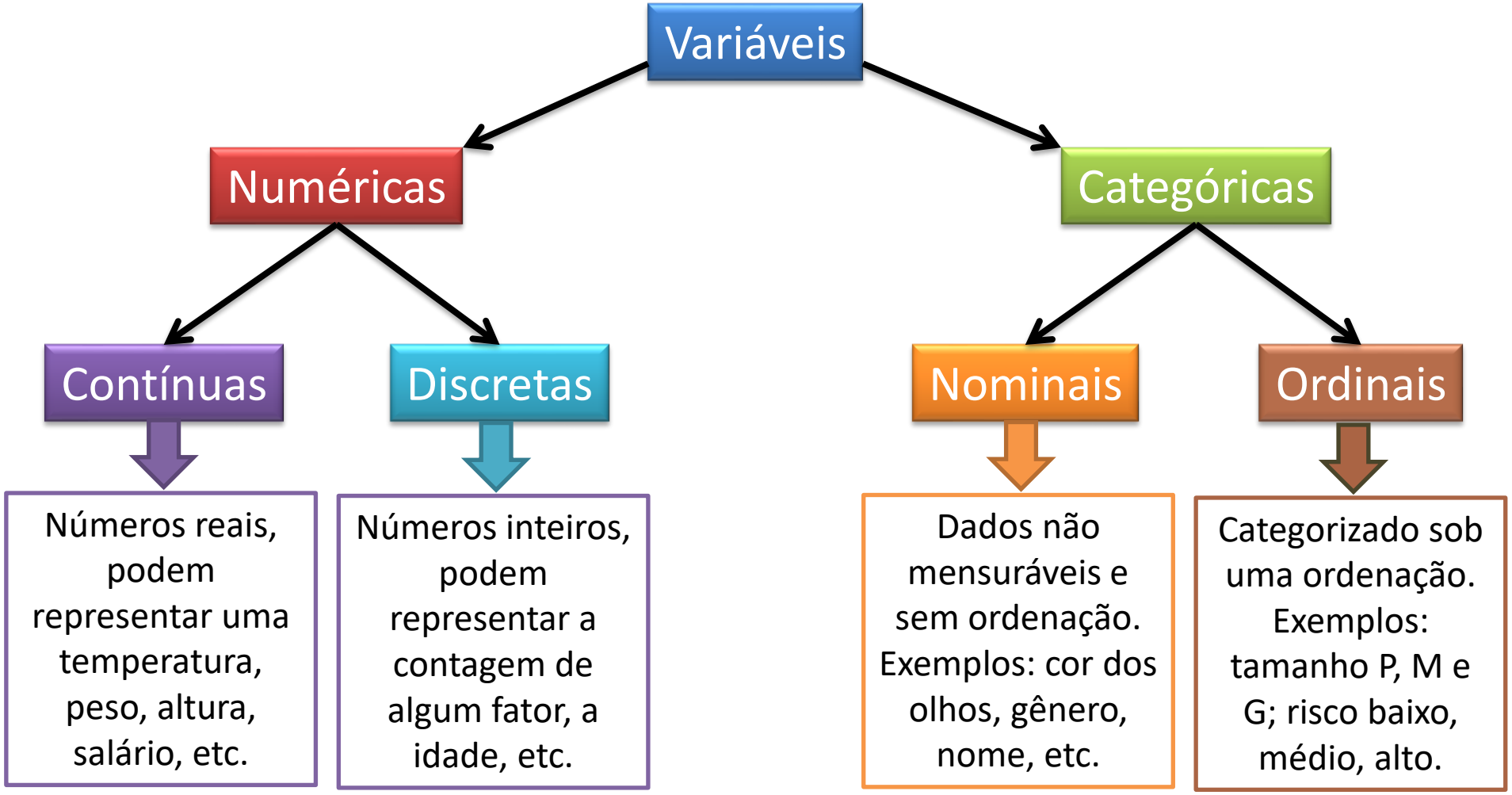
- **Tipos de variáveis**



- **Tipos de variáveis**



- **Tipos de variáveis**



  – Certos algoritmos podem aceitar apenas variáveis numéricas, logo, pode ser necessário **converter** uma variável categórica para numérica.

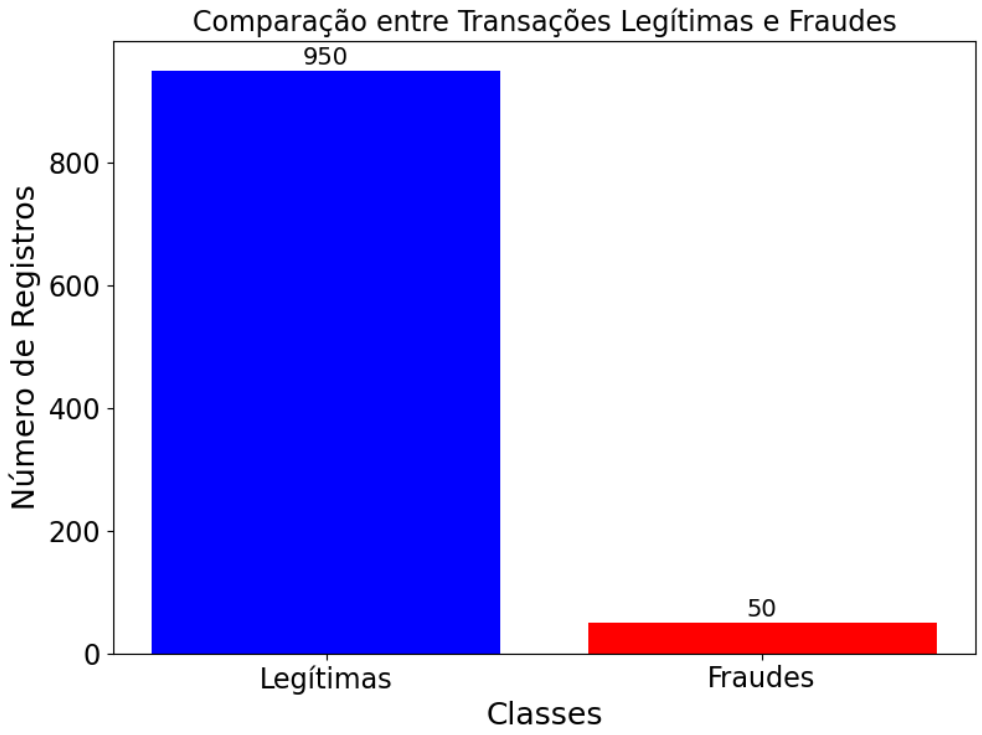
Pré-processamento de dados

- **Balanceamento de dados**
  - Em problemas de classificação, é comum que as classes não estejam igualmente representadas nos dados
  - **Dados desbalanceados** ocorrem quando uma ou mais classes possuem significativamente mais exemplos do que as outras.
  - Isso pode causar um **viés no modelo**, que pode aprender a **favorecer a classe majoritária**, resultando em baixa performance na classificação da classe minoritária.

Pré-processamento de dados

- **Balanceamento de dados**

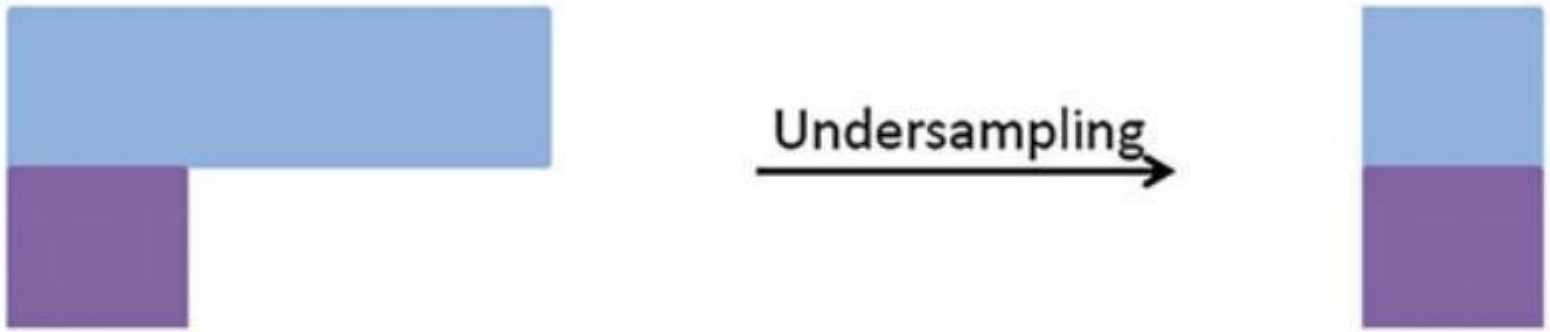
➢ **Exemplo:** Imagine um problema de **detecção de fraudes**, onde **95%** das transações são **legítimas** e apenas **5%** são **fraudulentas**. Um modelo desbalanceado pode aprender a sempre prever transações como **legítimas**, alcançando uma **alta acurácia**, mas falhando em detectar fraudes.



Pré-processamento de dados

- **Como balancear?**
  - Existem algumas maneiras para tratar este problema:

- **Undersampling (Subamostragem):** Reduz a representação da classe majoritária ao remover exemplos até que as classes estejam equilibradas.

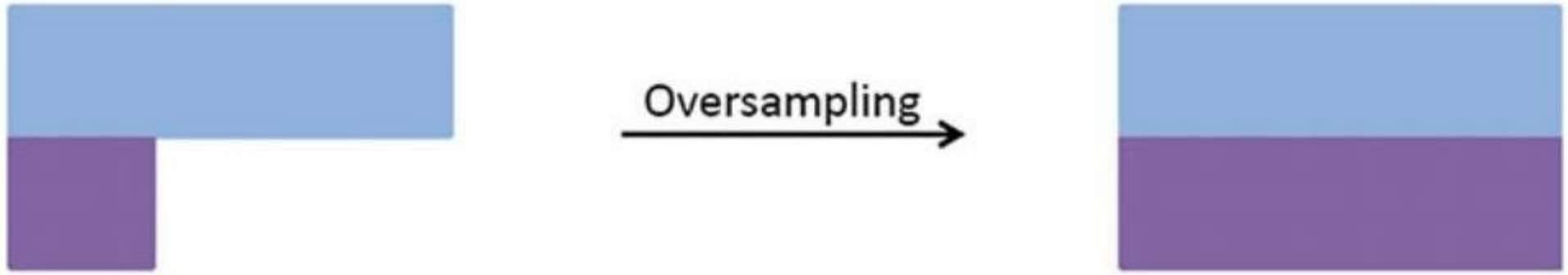


- Cuidado! Pode resultar na perda de informação, já que exemplos importantes da classe majoritária são removidos.

Pré-processamento de dados

- **Como balancear?**
  - Existem algumas maneiras para tratar este problema:

- **Oversampling (Superamostragem):** Aumenta a representação da classe minoritária replicando exemplos existentes ou gerando novos exemplos sintéticos.

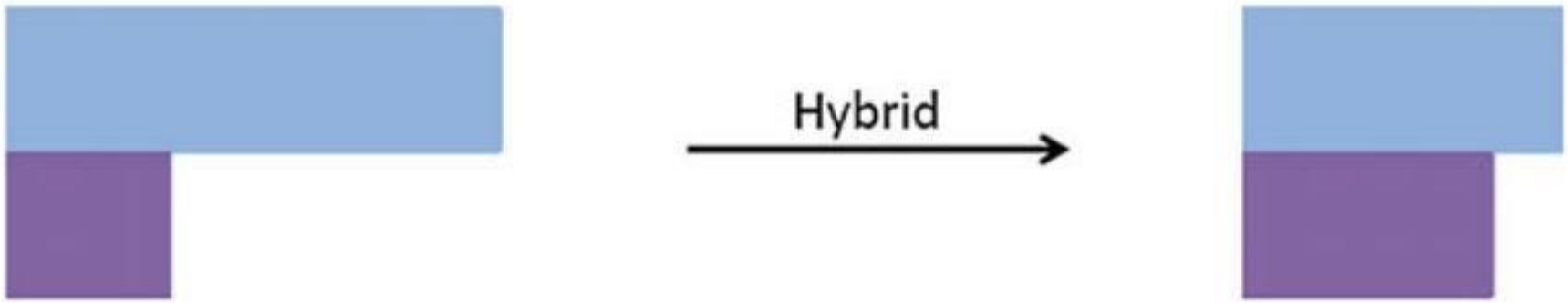


  - Cuidado! Oversampling pode levar a overfitting, pois o modelo pode "decorar" os exemplos repetidos.

Pré-processamento de dados

- **Como balancear?**
  - Existem algumas maneiras para tratar este problema:

- **Uso de técnicas híbridas:** Combina oversampling da classe minoritária e undersampling da classe majoritária para melhorar o desempenho do modelo.



- Cuidado! Ambas as desvantagens (e vantagens) poderão estar presentes!

Pré-processamento de dados

- **Base de dados de crédito**
  - Vamos utilizar uma base de dados que envolve a **classificação** do risco de conceder crédito para uma pessoa, baseando-se em alguns atributos.
  - A base de dados possui 1000 registros.
  - O objetivo do problema é classificar se conceder crédito para determinada pessoa é uma operação de **alto risco** ou de **baixo risco**.
  - Os dados são **reais** e fazem parte de uma base real de um banco alemão.

Fonte: https://www.kaggle.com/uciml/german-credit

Pré-processamento de dados

- **Base de dados de crédito**
  - A base possui **9 atributos** que servirão como parâmetros de **entrada** para o problema de classificação.
  - São eles:
    1. Idade
    2. Sexo (Masculino ou Feminino)
    3. Profissão (sem capacitação, baixa, média, alta)
    4. Moradia (própria, aluguel, gratuita)
    5. Poupança (pequena, moderada, alta, muito alta)
    6. Conta corrente (pequena, moderada, alta, muito alta)
    7. Valor do crédito solicitado
    8. Meses de duração do empréstimo
    9. Motivo do empréstimo (carro, equipamento, férias, outros)

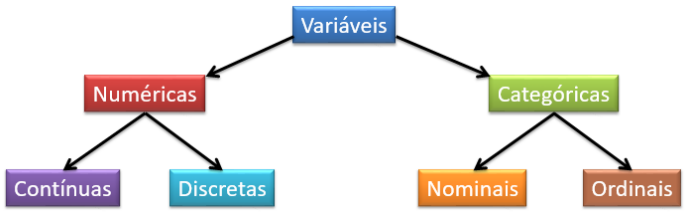
Pré-processamento de dados

- **Base de dados de crédito**
  - A **variável objetivo**, que corresponde à classe que desejamos predizer para cada registro, é representado por:

10. Risco de crédito (alto ou baixo)

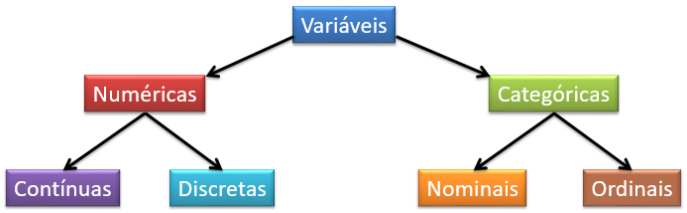
  - Assim, temos **9 características** que servirão como entrada para nossos algoritmos de aprendizado de máquina.
  - Os algoritmos irão, a partir delas, **procurar padrões** que permitirão a previsão da décima característica: o risco de crédito.

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



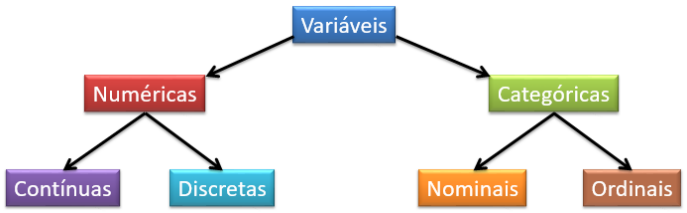
1. Idade
2. Sexo (Masculino ou Feminino)
3. Profissão (sem capacitação, baixa, média, alta)
4. Moradia (própria, aluguel, gratuita)
5. Poupança (pequena, moderada, alta, muito alta)
6. Conta corrente (pequena, moderada, alta, muito alta)
7. Valor do crédito solicitado
8. Meses de duração do empréstimo
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



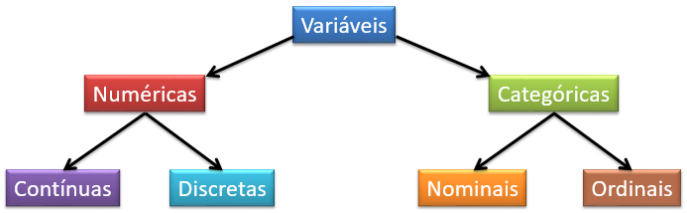
1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino)
3. Profissão (sem capacitação, baixa, média, alta)
4. Moradia (própria, aluguel, gratuita)
5. Poupança (pequena, moderada, alta, muito alta)
6. Conta corrente (pequena, moderada, alta, muito alta)
7. Valor do crédito solicitado
8. Meses de duração do empréstimo
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



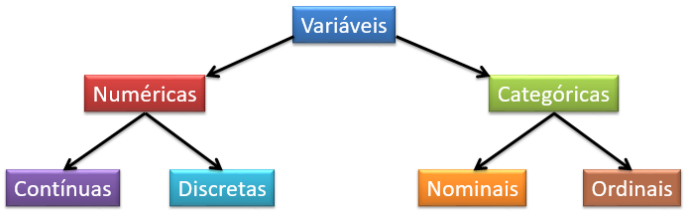
1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta)
4. Moradia (própria, aluguel, gratuita)
5. Poupança (pequena, moderada, alta, muito alta)
6. Conta corrente (pequena, moderada, alta, muito alta)
7. Valor do crédito solicitado
8. Meses de duração do empréstimo
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



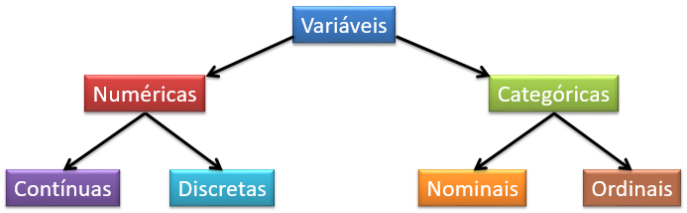
1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**
4. Moradia (própria, aluguel, gratuita)
5. Poupança (pequena, moderada, alta, muito alta)
6. Conta corrente (pequena, moderada, alta, muito alta)
7. Valor do crédito solicitado
8. Meses de duração do empréstimo
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



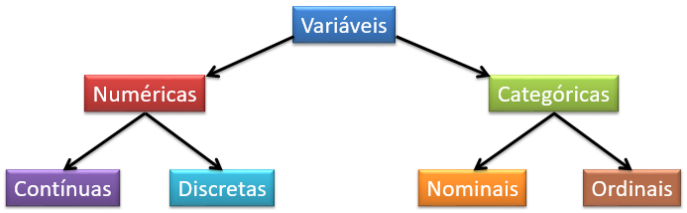
1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**
4. Moradia (própria, aluguel, gratuita) – **Categórica nominal**
5. Poupança (pequena, moderada, alta, muito alta)
6. Conta corrente (pequena, moderada, alta, muito alta)
7. Valor do crédito solicitado
8. Meses de duração do empréstimo
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



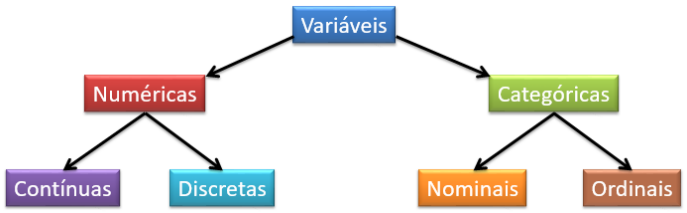
1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**
4. Moradia (própria, aluguel, gratuita) – **Categórica nominal**
5. Poupança (pequena, moderada, alta, muito alta) – **Categórica ordinal**
6. Conta corrente (pequena, moderada, alta, muito alta)
7. Valor do crédito solicitado
8. Meses de duração do empréstimo
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



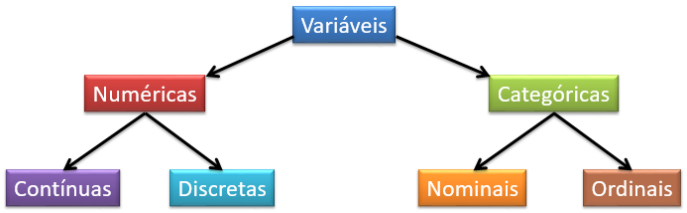
1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**
4. Moradia (própria, aluguel, gratuita) – **Categórica nominal**
5. Poupança (pequena, moderada, alta, muito alta) – **Categórica ordinal**
6. Conta corrente (peq., mod., alta, muito alta) **–** **Categórica ordinal**
7. Valor do crédito solicitado
8. Meses de duração do empréstimo
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



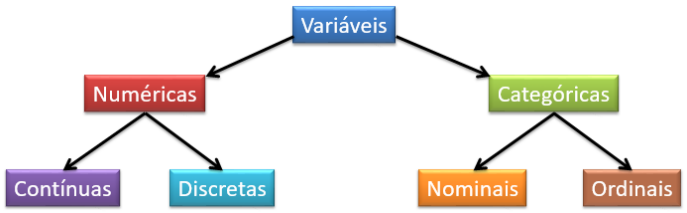
1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**
4. Moradia (própria, aluguel, gratuita) – **Categórica nominal**
5. Poupança (pequena, moderada, alta, muito alta) – **Categórica ordinal**
6. Conta corrente (peq., mod., alta, muito alta) **–** **Categórica ordinal**
7. Valor do crédito solicitado **–** **Numérica contínua**
8. Meses de duração do empréstimo
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



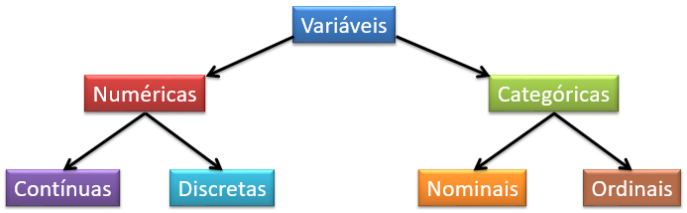
1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**
4. Moradia (própria, aluguel, gratuita) – **Categórica nominal**
5. Poupança (pequena, moderada, alta, muito alta) – **Categórica ordinal**
6. Conta corrente (peq., mod., alta, muito alta) **–** **Categórica ordinal**
7. Valor do crédito solicitado **–** **Numérica contínua**
8. Meses de duração do empréstimo – **Numérica discreta**
9. Motivo do empréstimo (carro, equipamento, férias, outros)
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**
4. Moradia (própria, aluguel, gratuita) – **Categórica nominal**
5. Poupança (pequena, moderada, alta, muito alta) – **Categórica ordinal**
6. Conta corrente (peq., mod., alta, muito alta) **–** **Categórica ordinal**
7. Valor do crédito solicitado **–** **Numérica contínua**
8. Meses de duração do empréstimo – **Numérica discreta**
9. Motivo do empréstimo (carro, outros) – **Categórica nominal**
10. Risco de crédito (alto ou baixo)

- **Análise dos tipos dos dados**
  - O primeiro passo do pré-processamento de dados é a análise dos tipos dos dados.



1. Idade – **Numérica discreta**
2. Sexo (Masculino ou Feminino) – **Categórica nominal**
3. Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**
4. Moradia (própria, aluguel, gratuita) – **Categórica nominal**
5. Poupança (pequena, moderada, alta, muito alta) – **Categórica ordinal**
6. Conta corrente (peq., mod., alta, muito alta) **–** **Categórica ordinal**
7. Valor do crédito solicitado **–** **Numérica contínua**
8. Meses de duração do empréstimo – **Numérica discreta**
9. Motivo do empréstimo (carro, outros) – **Categórica nominal**
10. Risco de crédito (alto ou baixo) – **Categórica nominal**

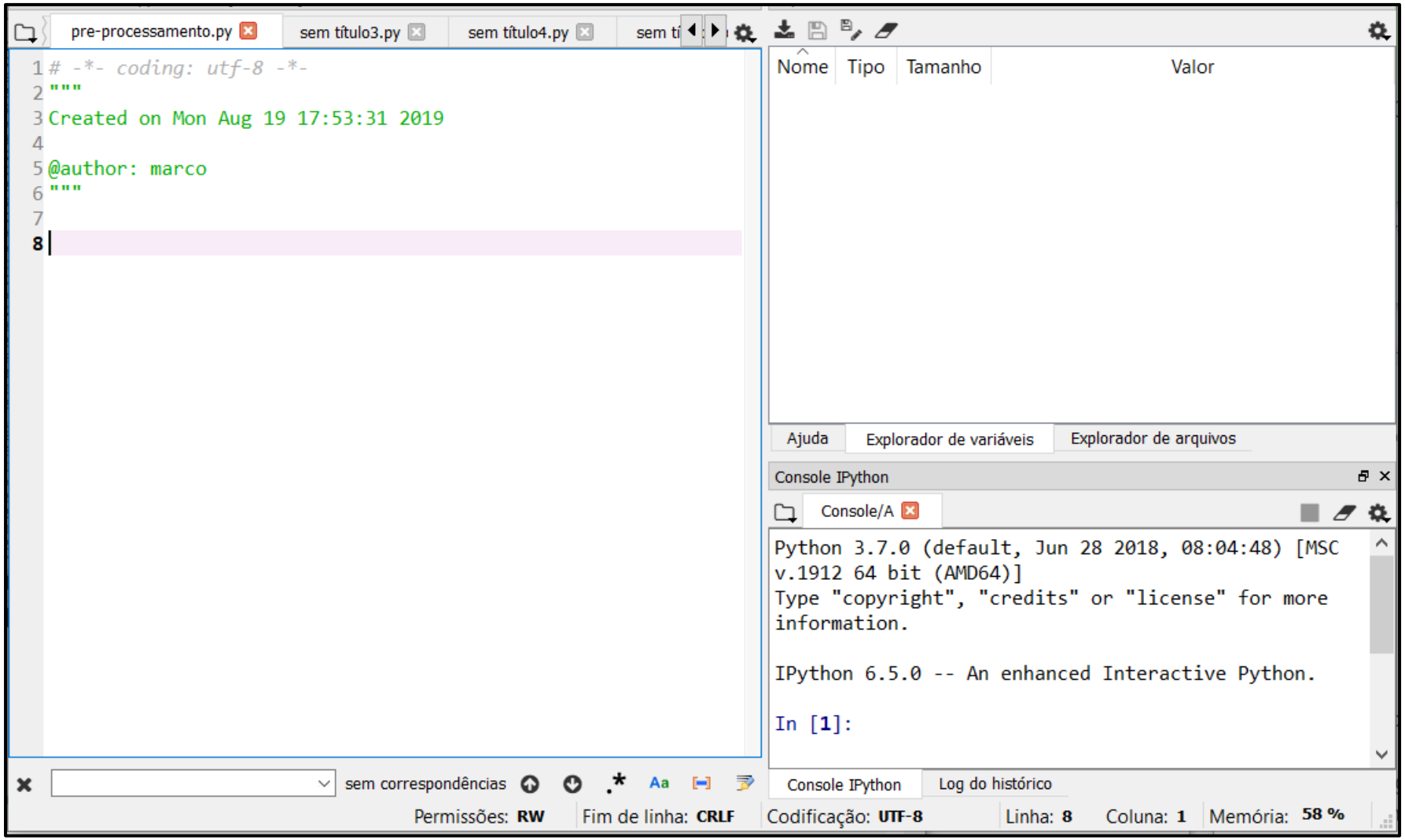
Pré-processamento de dados

- **Balanceamento dos dados**
  - A **variável objetivo**, que corresponde à classe que desejamos predizer para cada registro, é representado por:

10. Risco de crédito (alto ou baixo)

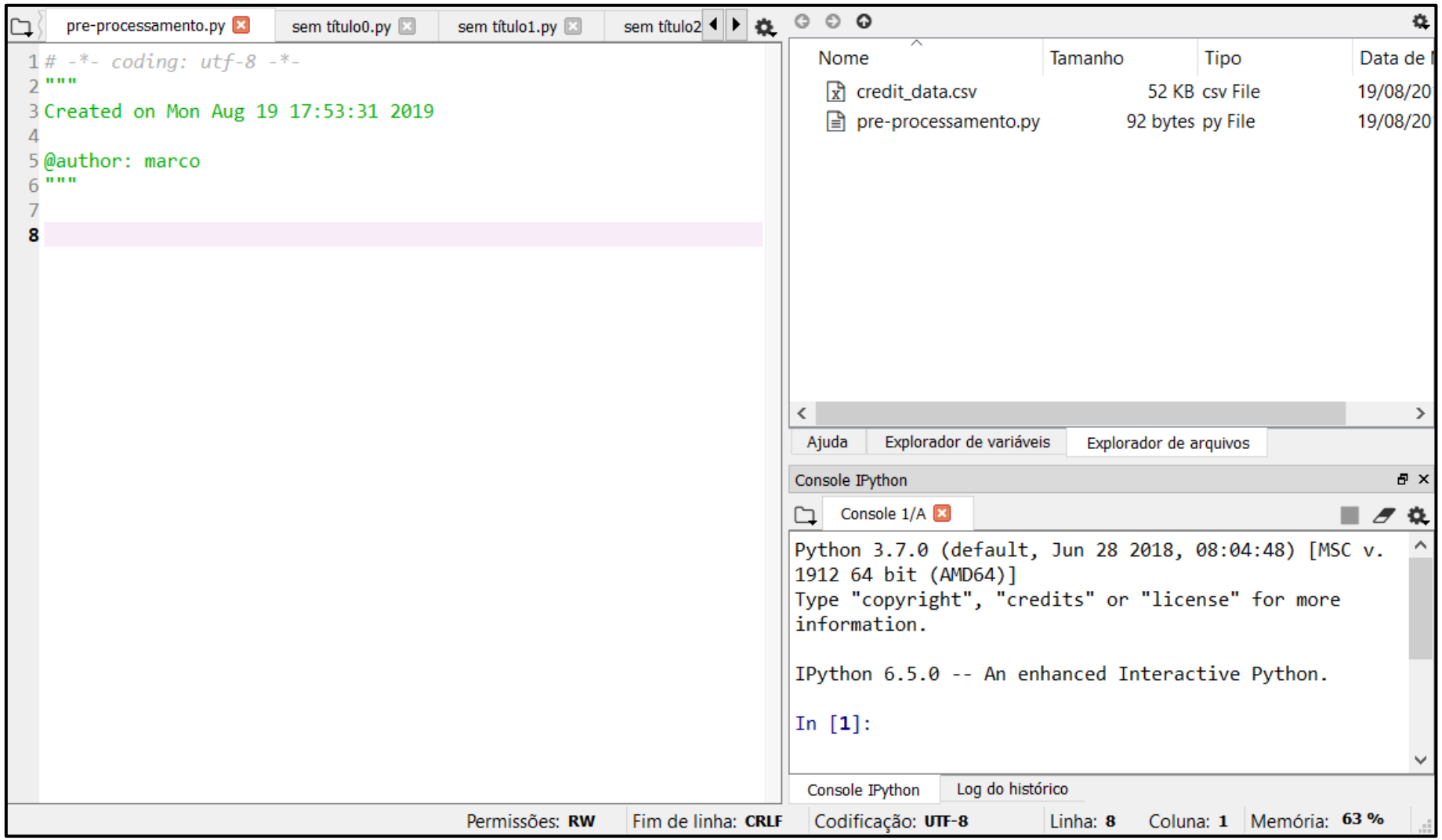
  - Assim, temos **9 características** que servirão como entrada para nossos algoritmos de aprendizado de máquina.
  - Os algoritmos irão, a partir delas, **procurar padrões** que permitirão a previsão da décima característica: o risco de crédito.

- **Abrindo a base de dados**
  - O próximo passo é abrir a base de dados com o pandas.
  - Abra o Anaconda, inicie o Spyder e crie um novo arquivo em branco:



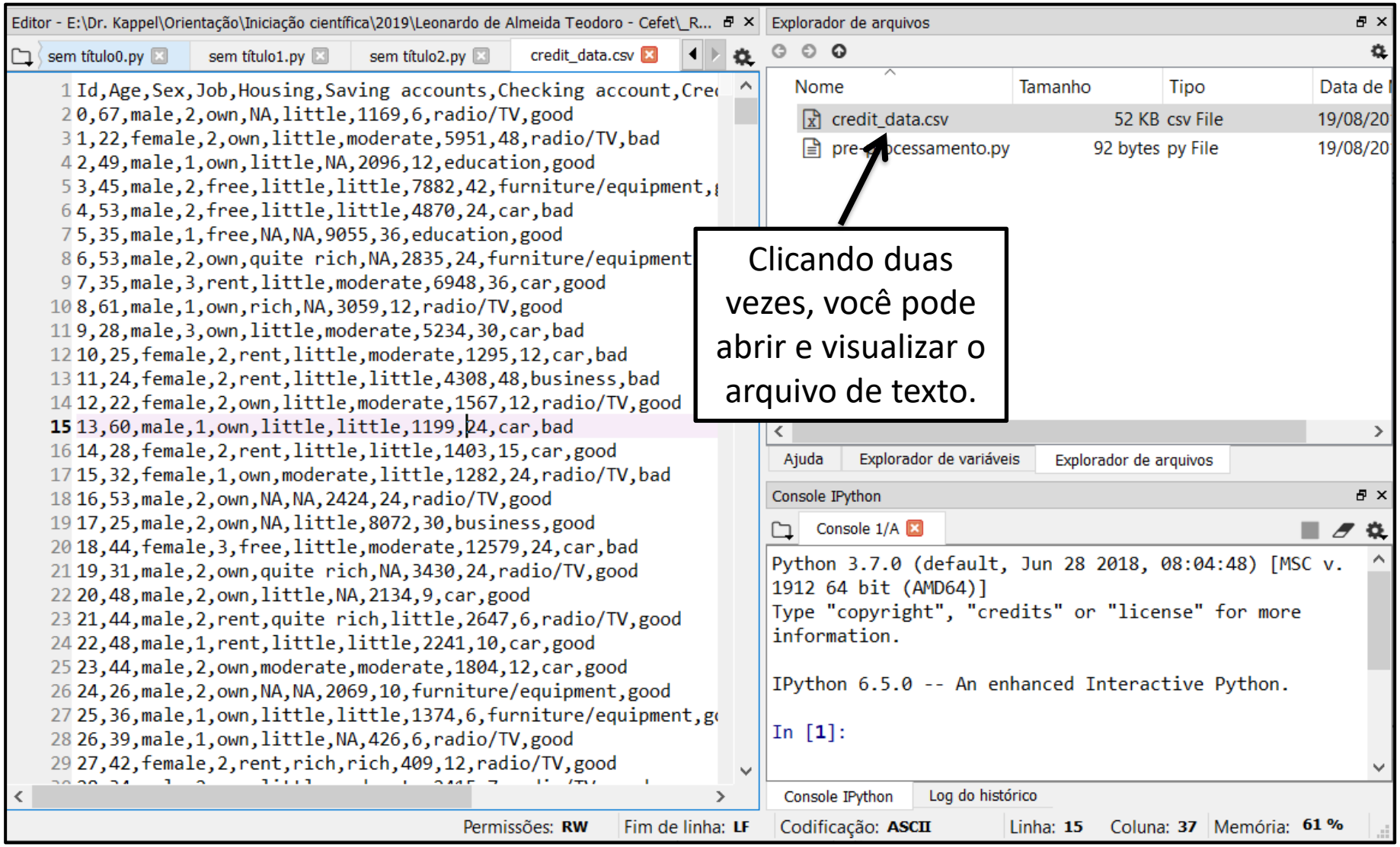
- **Abrindo a base de dados**

  – Salve o arquivo na mesma pasta em que está o arquivo credit_data.csv.

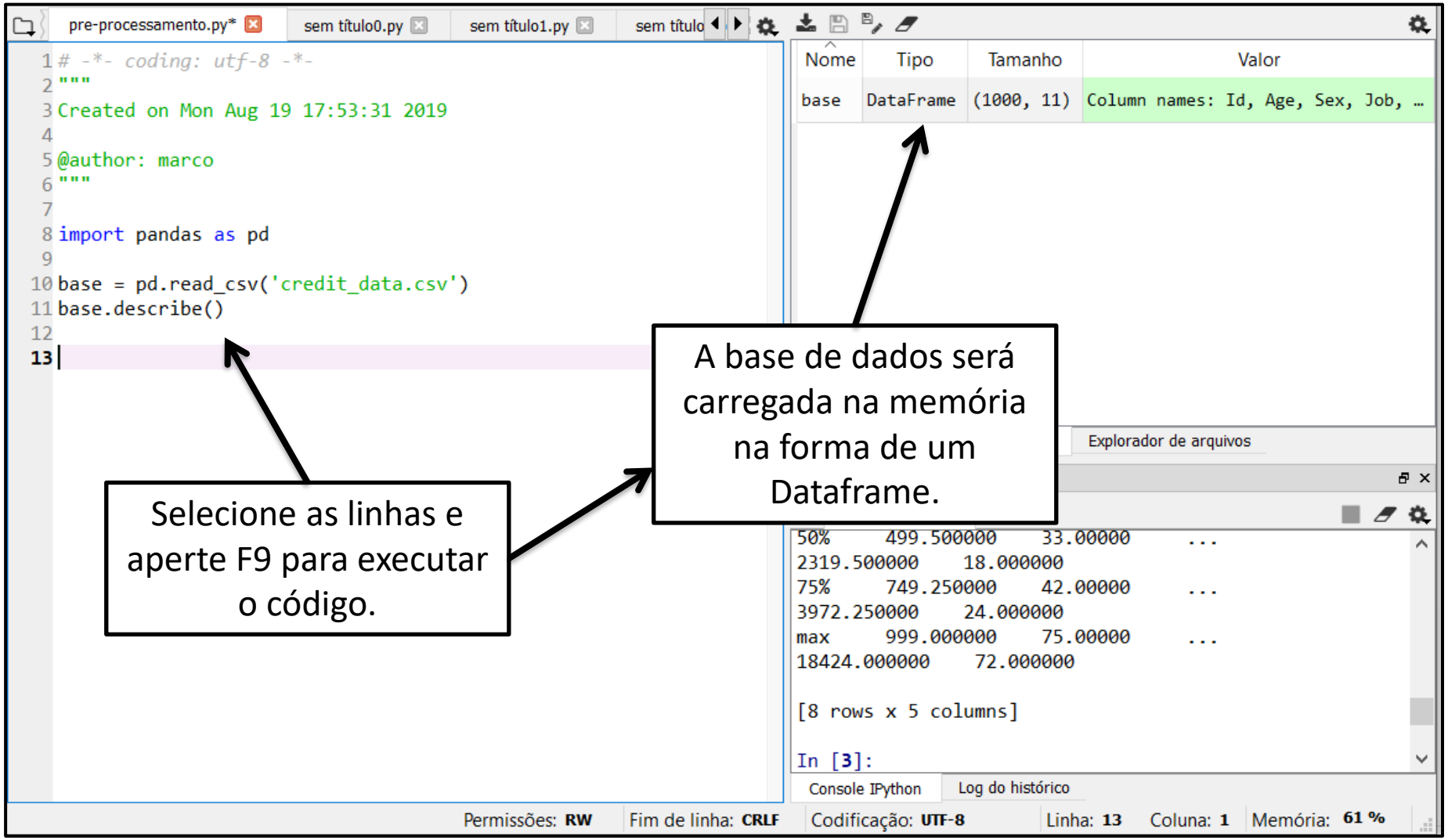


- **Abrindo a base de dados**

  – Salve o arquivo na mesma pasta em que está o arquivo credit_data.csv.

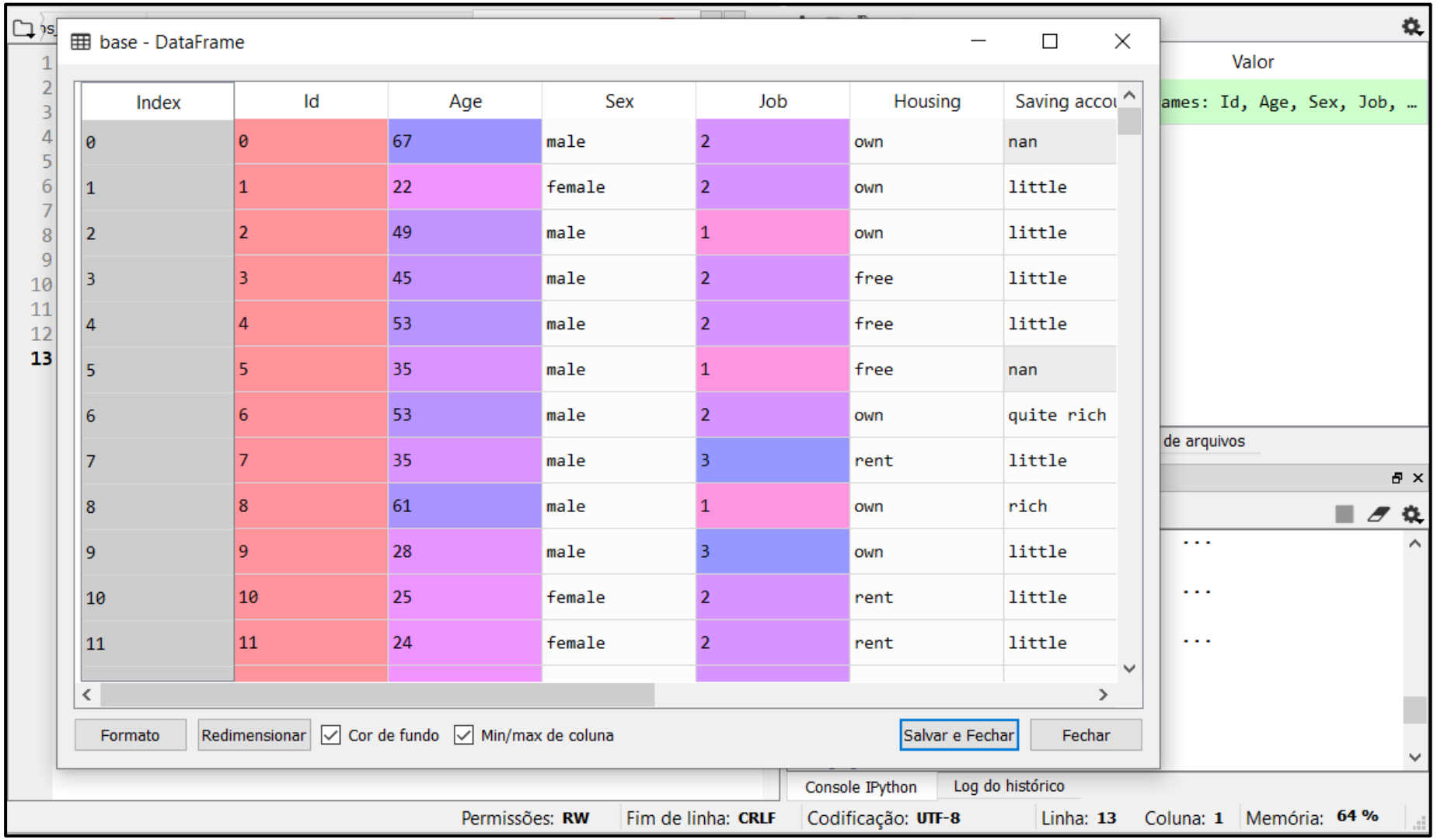


- **Abrindo a base de dados**
  - Digite o seguinte código:



- **Abrindo a base de dados**

  – O pandas permite que você visualize os dados na forma de uma planilha:



Index	Id	Age	Sex	Job	Housing	Saving accou
0	0	67	male	2	own	nan
1	1	22	female	2	own	little
2	2	49	male	1	own	little
3	3	45	male	2	free	little
4	4	53	male	2	free	little
5	5	35	male	1	free	nan
6	6	53	male	2	own	quite rich
7	7	35	male	3	rent	little
8	8	61	male	1	own	rich
9	9	28	male	3	own	little
10	10	25	female	2	rent	little
11	11	24	female	2	rent	little

- **Abrindo a base de dados**

  – O comando “describe” exibe as estatísticas básicas das variáveis numéricas da base de dados.

os_climaticos_RJ.py | pre-processamento.py*

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Aug 19 17:53:31 2019

@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()
```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(1000, 11)	Column names: Id, Age, Sex, Job, Housing, Saving accounts, Checking ac ...

Ajuda | Explorador de variáveis | Explorador de arquivos

Console IPython

Console 1/A

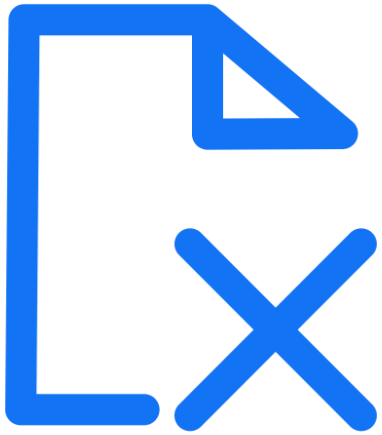
```
In [2]: import pandas as pd
   ...:
   ...: base = pd.read_csv('credit_data.csv')
   ...: base.describe()
Out[2]:
                Id         Age     ...       Credit amount     Duration
count  1000.000000  1000.00000     ...         1000.000000  1000.000000
mean    499.500000    35.36200     ...         3271.258000    20.903000
std     288.819436    11.93572     ...         2822.736876    12.058814
min       0.000000   -40.00000     ...          250.000000     4.000000
25%     249.750000    27.00000     ...         1365.500000    12.000000
50%     499.500000    33.00000     ...         2319.500000    18.000000
75%     749.250000    42.00000     ...         3972.250000    24.000000
max     999.000000    75.00000     ...        18424.000000    72.000000

[8 rows x 5 columns]

In [3]:
```

Console IPython | Log do histórico

Abrir arquivo | Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 11 | Coluna: 1 | Memória: 65 %

Pré-processamento de dados

- **Valores inconsistentes**
  - Como lidamos com grandes massas de dados reais, é comum encontrarmos dados inconsistentes.
  - Dados inconsistentes são valores que não condizem com o **significado** da variável.
  - Por exemplo:
    - Idades negativas
    - Categorias inexistentes
    - Dados inválidos em geral



- **Valores inconsistentes**
  - O primeiro passo é localizar os dados inconsistentes na planilha.
  - Para isso, usaremos o método .loc() do pandas.

pre-processamento.py*

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Aug 19 17:53:31 2019

@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()

base.loc[base['Age'] < 0]
```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(1000, 11)	Column names: Id, Age, Sex, Job, Housing, Saving accounts, Checking ac ...

Ajuda | Explorador de variáveis | Explorador de arquivos

Console IPython

Console 1/A

```
In [5]: base.loc[base['Age'] < 0]
Out[5]:
      Id  Age   Sex  Job  ...  Credit amount Duration   Purpose  Risk
39    39  -24  male    2  ...            458        9  radio/TV  good
89    89  -28  male    2  ...           1108       12   repairs   bad
786  786  -40  male    2  ...           2675       22  radio/TV  good

[3 rows x 11 columns]

In [6]:
```

Console IPython | Log do histórico

Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 13 | Coluna: 26 | Memória: 61 %

Este comando irá localizar todos os registros da base que possuem esta inconsistência.

Pré-processamento de dados

- **Valores inconsistentes**
  - Neste caso, existem **4 técnicas** que podemos aplicar.

1. Apagar a coluna

   - Caso sejam muitas ocorrências da inconsistência em uma mesma coluna, pode ser interessante **eliminá-la por completo**.
   - Deve ser feito com bastante **cautela**, pois a coluna pode representar alguma informação de grande importância.

- **Valores inconsistentes**

  – Para eliminar uma coluna com o Pandas, deve-se utilizar o comando "drop":

pre-processamento.py* | sem título0.py | sem título1.py

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Aug 19 17:53:31 2019

@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()
base.loc[base['Age'] < 0]

# Apagar a coluna
base.drop('Age', 1, inplace=True)

```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(1000, 10)	Column names: Id, Sex, Job, Housing, Saving accounts, Checking account ...

Ajuda | Explorador de variáveis | Explorador de arquivos

Console IPython

Console 1/A

```
    39  -24  male    2  ...          458        9
dio/TV  good
    89  -28  male    2  ...         1108       12
pairs   bad
6  786  -40  male    2  ...         2675       22
radio/TV  good

[3 rows x 11 columns]

In [6]: base.drop('Age', 1, inplace=True)

In [7]:
```

Console IPython | Log do histórico

Perceba que o número total de colunas do Dataframe foi reduzido.

Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 17 | Coluna: 1 | Memória: 61 %

Pré-processamento de dados

- **Valores inconsistentes**
  - Neste caso, existem 4 técnicas que podemos aplicar.

2. Apagar apenas os registros com problema

   - Todas as **pessoas** que estão com o dado errado serão eliminadas da base.
   - Ao invés de termos 1000 registros, passaremos a considerar apenas 997.
   - Tem **menos impacto**, mas as outras informações destas pessoas poderiam ser relevantes para o problema.

- **Valores inconsistentes**

  – Para eliminar uma linha com o Pandas, também deve-se utilizar o comando “drop”:

pre-processamento.py | sem título0.py | sem título1.py

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Aug 19 17:53:31 2019

@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()
base.loc[base['Age'] < 0]

# Apagar a coluna
base.drop('Age', 1, inplace=True)

# Apagar apenas os registros com problema
base.drop(base[base.Age < 0].index, inplace=True)
```

Nome	Tipo	Tamanho	Valor
base	DataFrame	(997, 11)	Column names: Id, Age, Sex, Job, Housing, Saving accounts, Checking ac ...

Ajuda | Explorador de variáveis | Explorador de arquivos

Console IPython

```
File "<ipython-input-10-e51aa590c3f0>", line 1, in <module>
  base.drop(base[base.Age < 0].index, inplace=True)

: drop() got an unexpected keyword argument

In [11]: base.drop(base[base.Age < 0].index, inplace=True)

In [12]:
```

Console IPython | Log do histórico

Perceba que o número total de linhas do Dataframe foi reduzido.

Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 18 | Coluna: 50 | Memória: 60 %

Pré-processamento de dados

- **Valores inconsistentes**
  - Neste caso, existem 4 técnicas que podemos aplicar.

3. Preencher os valores manualmente

   - Consiste na **troca manual** dos valores inválidos pelos valores verdadeiros.
   - Pode não ser viável, pela dificuldade de se **rastrear** os dados reais.
   - Em geral, apenas é possível quando se possui os dados pessoais dos indivíduos envolvidos, como nome e contato.

Pré-processamento de dados

- **Valores inconsistentes**
  - Neste caso, existem 4 técnicas que podemos aplicar.

4. Preencher os valores com a média (ou o mais frequente).

   - Técnica mais **eficiente**.
   - Causa **pouco impacto** nos algoritmos e não perde a informação dos outros atributos, que pode ser valiosa.
   - Basta calcular o **valor médio** do atributo e substituí-lo nos registros inconsistentes.

- **Valores inconsistentes**
  - Deve-se tomar o cuidado de não considerar os dados inconsistentes no cálculo da média.

pre-processamento.py* | sem título0.py | sem título1.py

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Aug 19 17:53:31 2019

@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()
base.loc[base['Age'] < 0]

# Apagar a coluna
base.drop('Age', 1, inplace=True)

# Apagar apenas os registros com problema
base.drop(base[base.Age < 0].index, inplace=True)

# Preencher os valores com a média
age_media = base['Age'][base.Age > 0].mean()
base.loc[base.Age < 0, 'Age'] = age_media
```

Nome	Tipo	Tamanho	Valor
age_media	float	1	35.56068204613842
base	DataFrame	(1000, 11)	Column names: Id, Age, Sex, Job, Hous…

Ajuda | Explorador de variáveis | Explorador de arquivos

Console IPython

Console 1/A

```
In [16]: age_media = base['Age'][base.Age > 0].mean()

In [17]: base.loc[base.Age < 0, 'Age'] = age_media

In [18]: base.loc[base['Age'] < 0]
Out[18]:

Housing, Saving accounts,
t, Duration, Purpose, Risk]
```

Calcula a média e substitui apenas os registros que apresentam a inconsistência.

Permissões: RW | Linha: 24 | Coluna: 1 | Memória: 59 %

Pré-processamento de dados

- **Valores faltantes**
  - Outro tipo de erro comum em bases de dados reais é a **ausência** de alguns dados.
  - Este tipo de problema ocorre quando algum campo **não foi preenchido** em algum formulário, ou algum dado foi **omitido** em algum momento.
  - Por exemplo:
    - Qualquer **campo vazio** em um registro, mas preenchido nos demais.

Pré-processamento de dados

- **Valores faltantes**
  - As mesmas 4 técnicas podem ser aplicadas para resolver o problema dos valores faltantes:

1. Apagar a coluna
2. Apagar apenas os registros com problema
3. Preencher os valores manualmente
4. **Preencher os valores com a média (ou o mais frequente)**

Em geral, a estratégia mais eficiente.

- **Valores faltantes**

  – Primeiro, é necessário identificar quais são as colunas que apresentam o problema:

pre-processamento.py* | sem título0.py | sem título1.py | sem título2.py

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Aug 19 17:53:31 2019

@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()
base.loc[base['Age'] < 0]

# Apagar a coluna
base.drop('Age', 1, inplace=True)

# Apagar apenas os registros com problema
base.drop(base[base.Age < 0].index, inplace=True)

# Preencher os valores com a media
age_media = base['Age'][base.Age > 0].mean()
base.loc[base.Age < 0, 'Age'] = age_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()
```

Nome	Tipo	Tamanho	Valor
age_media	float	1	35.56068204613842
base	DataFrame	(1000, 11)	Column names: Id, …

Ajuda | Explorador de variáveis | Explorador de arquivos

Console IPython

Console 1/A

```
In [35]: pd.isnull(base).any()
Out[35]:
Id                  False
Age                 False
Sex                 False
Job                 False
Housing             False
Saving accounts      True
Checking account     True
Credit amount       False
Duration            False
Purpose             False
Risk                False
dtype: bool

In [36]:
```

Console IPython | Log do histórico

Estas colunas possuem algum valor nulo ou NaN.

Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 25 | Coluna: 22 | Memória: 63 %

- **Valores faltantes**

  – Como se trata de uma variável categórica, precisamos identificar o **valor mais frequente** e inserir nos locais com dados faltantes:

pre-processamento.py* | sem título0.py | sem título1.py | sem título2.py | tratamento_de_dados.py - exemplo\

```python
@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()
base.loc[base['Age'] < 0]

# Apagar a coluna
base.drop('Age', 1, inplace=True)

# Apagar apenas os registros com problema
base.drop(base[base.Age < 0].index, inplace=True)

# Preencher os valores com a média
age_media = base['Age'][base.Age > 0].mean()
base.loc[base.Age < 0, 'Age'] = age_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts'])].describe().top
base.loc[pd.isnull(base['Saving accounts']), 'Saving accounts'] = saving_maioria
```

Salva o valor mais frequente e insere nas posições nulas ou NaN.

Nome	Tipo	Tamanho
age_media	float	1
base	DataFrame	(1000, 11)
saving_maioria	str	1

Aj... | Explorador de... | Explorador de...

Console IPython

Console 1/A

```
ittlemoderatelittlelittlelitt
lequite
richlittlelittlelittlelittlel
ittlemoderatelittlelittlelitt
lelittlelittlelittlelittlelit
tlelittlelittlerichlittlelitt
lelittlemoderatemoderatelittl
equite richquite
richlittlelittlemoderatelittl
elittlelittlelittlelittlemode
ratemoderatemoderaterichlittl
emoderatemoderatelittlequite
richlittlelittlelittlelittleq
```

Console IPython | Log do histórico

Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 28 | Coluna: 93 | Memória: 65 %

- **Valores faltantes**
  - O mesmo deve ser feito para a coluna 'Checking account':

pre-processamento.py | sem título0.py | sem título1.py | sem título2.py | tratamento_de_dados.py - exemplo\...\Pré-

```python
@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()
base.loc[base['Age'] < 0]

# Apagar a coluna
base.drop('Age', 1, inplace=True)

# Apagar apenas os registros com problema
base.drop(base[base.Age < 0].index, inplace=True)

# Preencher os valores com a média
age_media = base['Age'][base.Age > 0].mean()
base.loc[base.Age < 0, 'Age'] = age_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts'])].describe().top
base.loc[pd.isnull(base['Saving accounts']), 'Saving accounts'] = saving_maioria

# Preencher os valores com o mais frequente
checking_maioria = base['Checking account'][pd.notnull(base['Checking account'])].describe().top
base.loc[pd.isnull(base['Checking account']), 'Checking account'] = checking_maioria
```

Nome	Tipo	Tama
age_media	float	1
base	DataFrame	(1000,
checking_maioria	str	1
saving_maioria	str	1

Aj... | Explorador d... | Explorador d...

Console IPython

Console 1/A

```
In [71]:
```

Console IPython | Log do histórico

Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 34 | Coluna: 1 | Memória: 64 %

- **Valores faltantes**
  - Repetindo o teste, vemos que não existem mais valores faltantes na base.

pre-processamento.py | sem título0.py | sem título1.py | sem título2.py | tratamen

```python
@author: marco
"""

import pandas as pd

base = pd.read_csv('credit_data.csv')
base.describe()
base.loc[base['Age'] < 0]

# Apagar a coluna
base.drop('Age', 1, inplace=True)

# Apagar apenas os registros com problema
base.drop(base[base.Age < 0].index, inplace=True)

# Preencher os valores com a média
age_media = base['Age'][base.Age > 0].mean()
base.loc[base.Age < 0, 'Age'] = age_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts']
base.loc[pd.isnull(base['Saving accounts']), 'Saving accounts'] = saving_ma

# Preencher os valores com o mais frequente
checking_maioria = base['Checking account'][pd.notnull(base['Checking accou
base.loc[pd.isnull(base['Checking account']), 'Checking account'] = checkin
```

Nome	Tipo	Tamanho	Valor
age_media	float	1	35.56068204613…
base	DataFrame	(1000, 11)	Column names: …

Ajuda | Explorador de variáveis | Explorador de arquivos

Console IPython

Console 1/A

```
account'][pd.notnull(base['Checking
account'])].describe().top
    ...: base.loc[pd.isnull(base['Checking
account']), 'Checking account'] = checking_maioria

In [71]: pd.isnull(base).any()
Out[71]:
Id                 False
Age                False
Sex                False
Job                False
Housing            False
Saving accounts    False
Checking account   False
Credit amount      False
Duration           False
Purpose            False
Risk               False
dtype: bool

In [72]:
```

Console IPython | Log do histórico

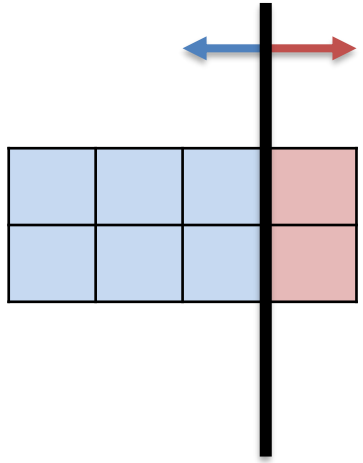
Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 26 | Coluna: 1 | Memória: 65 %

Pré-processamento de dados

- **Separação de variáveis**
  - Quando temos a intenção de aplicar técnicas de **aprendizado supervisionado**, precisamos separar a base de dados em duas categorias.

➢ **Variáveis previsoras**: dados que serão processados pelo algoritmo com o objetivo de calcular o valor do atributo classe.

➢ **Atributo(s) classe**: Variável objetivo, aquela cujo comportamento desejamos predizer.



- **Separação de variáveis**
  - No código, devemos separar os dados em previsores e classe (obs: id não é usado, pois não importa para a classificação):

```python
#base.drop('Age', 1, inplace=True)

# Apagar apenas os registros com problema
#base.drop(base[base.Age < 0].index, inplace=True)

# Preencher os valores com a média
age_media = base['Age'][base.Age > 0].mean()
base.loc[base.Age < 0, 'Age'] = age_media

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# Preencher os valores com o mais frequente
saving_maioria = base['Saving accounts'][pd.notnull(base['Saving accounts'])
base.loc[pd.isnull(base['Saving accounts']), 'Saving accounts'] = saving_ma

# Preencher os valores com o mais frequente
checking_maioria = base['Checking account'][pd.notnull(base['Checking accou
base.loc[pd.isnull(base['Checking account']), 'Checking account'] = checkin

# Separando dados em previsores e classes
cols_previsores = ['Age','Sex','Job','Housing', 'Saving accounts',
                   'Checking account', 'Credit amount', 'Duration',
                   'Purpose']
cols_classe = ['Risk']

previsores = base[cols_previsores]
classe = base[cols_classe]
```

Deve-se inserir os nomes das colunas correspondentes.

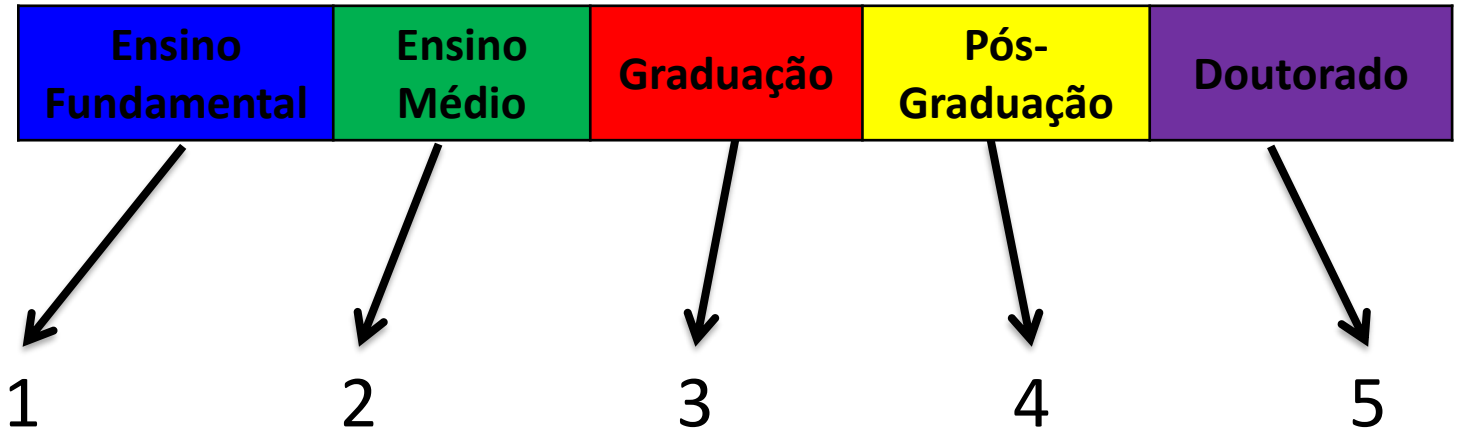
Nome	Tipo	Tamanho	Valor
a	DataFrame	(8, 5)	Column names: Id,…
age_media	float	1	35.56068204613842
base	DataFrame	(1000, 11)	Column names: Id,…
[illegible]	str	1	little
[illegible]	DataFrame	(1000, 1)	Column names: Risk
[illegible]	list	1	['Risk']
cols_previsores	list	9	['Age', 'Sex', 'J…
previsores	DataFrame	(1000, 9)	Column names: Age…
saving_maioria	str	1	little

```
In [84]:
```

Permissões: RW | Fim de linha: CRLF | Codificação: UTF-8 | Linha: 46 | Coluna: 1 | Memória: 62 %

Pré-processamento de dados

- **Transformação de variáveis categóricas**
  - A maioria dos algoritmos de Aprendizagem de Máquina são baseados em **cálculos** e **equações**.
  - Por este motivo, é necessário saber como **transformar rapidamente** as variáveis **categóricas** em variáveis **numéricas**.
  - Exemplo: variável “escolaridade”:



Pré-processamento de dados

- **Transformação de variáveis categóricas**
  - A base que estamos estudando possui as seguintes **variáveis categóricas**:

➢ Sexo (Masculino ou Feminino) – **Categórica nominal**

➢ Profissão (sem capacitação, baixa, média, alta) – **Categórica ordinal**

➢ Moradia (própria, aluguel, gratuita) – **Categórica nominal**

➢ Poupança (pequena, moderada, alta, muito alta) – **Categórica ordinal**

➢ Conta corrente (peq., mod., alta, muito alta) **–** **Categórica ordinal**

➢ Motivo do empréstimo (carro, outros) – **Categórica nominal**

➢ Risco de crédito (alto ou baixo) – **Categórica nominal**

- **Transformação de variáveis categóricas**

  – No código, precisamos transformar cada uma delas em variáveis numéricas:

```python
# ==============================================================================
#      Transformar as variáveis categóricas (ordinais) em valores numéricos
# ==============================================================================

from sklearn.preprocessing import LabelEncoder
import numpy as np

# Instanciar LabelEncoder para cada variável categórica
labelencoder_sex = LabelEncoder()
labelencoder_housing = LabelEncoder()
labelencoder_purpose = LabelEncoder()
labelencoder_saving_accounts = LabelEncoder()
labelencoder_checking_account = LabelEncoder()
labelencoder_risk = LabelEncoder()

# Codificação da variável 'Sex' (assumindo 'male' e 'female')
previsores.loc[:, 'Sex'] = labelencoder_sex.fit_transform(previsores.loc[:, 'Sex']) # female=0, male=1 (ordem alfabética)
previsores['Sex'] = previsores['Sex'].astype('int64')

# Codificação da variável 'Housing' (opcional, se houver)
# previsores.loc[:, 'Housing'] = labelencoder_housing.fi                    oc[:, 'Housing'])
# previsores['Housing'] = previsores['Housing'].astype('int64')

# Codificação ordinal para 'Saving accounts'
labelencoder_saving_accounts.classes_ = np.array(['little', 'moderate', 'rich', 'quite rich'])
previsores.loc[:, 'Saving accounts'] = labelencoder_saving_accounts.transform(previsores.loc[:, 'Saving accounts'])
previsores['Saving accounts'] = previsores['Saving accounts'].astype('int64')

# Codificação ordinal para 'Checking account'
labelencoder_checking_account.classes_ = np.array(['little', 'moderate', 'rich', 'quite rich'])
previsores.loc[:, 'Checking account'] = labelencoder_checking_account.transform(previsores.loc[:, 'Checking account'])
previsores['Checking account'] = previsores['Checking account'].astype('int64')

# Codificação da variável 'Purpose' (opcional, se houver)
# previsores.loc[:, 'Purpose'] = labelencoder_purpose.fit_transform(previsores.loc[:, 'Purpose']).astype('int64')
# previsores['Purpose'] = previsores['Purpose'].astype('int64')

# Codificação da variável alvo 'Risk' (assumindo 'good' e 'bad')
classe.loc[:, 'Risk'] = labelencoder_risk.fit_transform(classe.loc[:, 'Risk']) # bad=0, good=1 (ordem alfabética)
classe['Risk'] = classe['Risk'].astype('int64')
```

O LabelEncoder faz o trabalho de mapear as categorias em números.

Pré-processamento de dados

- **Transformação de variáveis categóricas**
  - Observe que algumas destas variáveis são categóricas **nominais**.
  - Para este tipo específico de variável, não faz sentido criar uma categorização numérica **sequencial**.

➢ Sexo (Masculino ou Feminino) – **Categórica nominal**

➢ Moradia (própria, aluguel, gratuita) – **Categórica nominal**

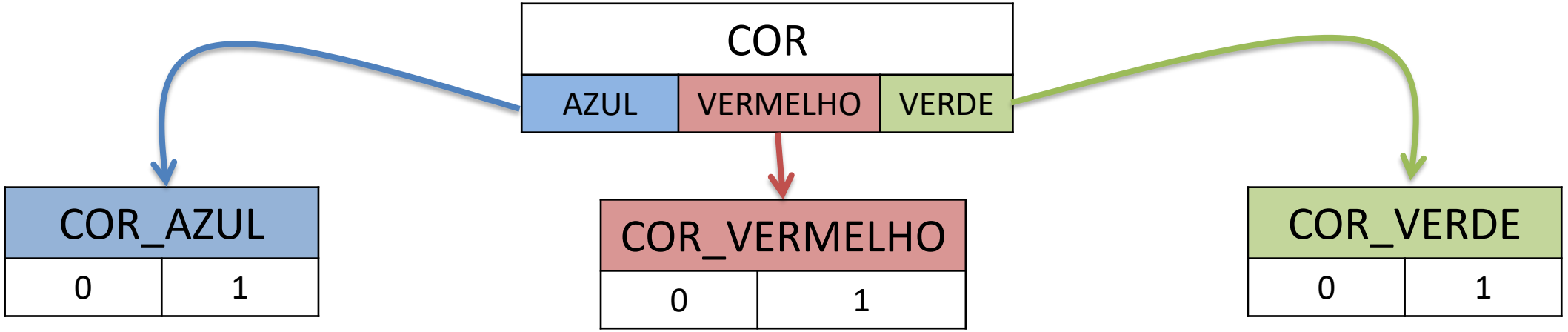
➢ Motivo do empréstimo (carro, outros) – **Categórica nominal**

➢ Risco de crédito (alto ou baixo) – **Categórica nominal**

  - Não podemos, por exemplo, achar que o sexo masculino é um valor "superior" ou "inferior" ao feminino.

Pré-processamento de dados

- **Transformação de variáveis categóricas**
  - O ideal, nesses casos, é **separar** a variável em diferentes variáveis "dummy".
  - Cada uma das **novas variáveis** terá apenas dois valores possíveis: 1 ou 0.
  - Isso dará a **mesma importância** nos cálculos para todos os valores das variáveis categóricas.



- **Transformação de variáveis categóricas**
  - No código, precisamos aplicar a conversão em cada variável categórica nominal:

Perceba que o número de atributos previsores passou para 18.

Spyder (Python 3.12)

Arquivo Editar Pesquisar Código Executar Depurar Consoles Projetos Ferramen

E:\PythonWorkspace\2025.1\aula3\pre-processamento.py

primeiroScript.py × pre-processamento.py × balanceamento.py ×

```python
classe['Risk'] = classe['Risk'].astype('int64')

# ==============================================================================
#      Transformar as variáveis categóricas (nominais) em variáveis dummy
# ==============================================================================

from sklearn.preprocessing import LabelBinarizer
labelbinarizer = LabelBinarizer()

# Variavel Purpose
variaveis_dummy = labelbinarizer.fit_transform(previsores['Purpose'])
novas_variaveis_dummy = labelbinarizer.classes_
df_variaveis_dummy = pd.DataFrame(variaveis_dummy, columns=novas_variaveis_dummy)
previsores = previsores.join(df_variaveis_dummy)
previsores = previsores.drop('Purpose',axis=1)

# Variavel Housing
variaveis_dummy = labelbinarizer.fit_transform(previsores['Housing'])
novas_variaveis_dummy = labelbinarizer.classes_
df_variaveis_dummy = pd.DataFrame(variaveis_dummy, columns=novas_variaveis_dummy)
previsores = previsores.join(df_variaveis_dummy)
previsores = previsores.drop('Housing',axis=1)

cols_previsores = previsores.columns
```

ythonWorkspace\2025.1\aula3

Nome	Tipo	Tamanho	
labelencoder_sex	preprocessing…	1	Lab
novas_variaveis_dummy	Array of str1…	(3,)	nda
previsores	DataFrame	(1000, 18)	Col
resumo	DataFrame	(8, 5)	Col
saving_maioria	str	6	lit
variaveis_dummy	Array of int32	(1000, 3)	[[0 [0

Ajuda Explorador de variáveis Figuras Arquivos

Console 1/A ×

```
pd.DataFrame(variaveis_dummy,
columns=novas_variaveis_dummy)
    ...: previsores = previsores.join(df_variaveis_dummy)
    ...: previsores = previsores.drop('Housing',axis=1)
    ...:
    ...: cols_previsores = previsores.columns

In [29]:
```

Console IPython Histórico

conda: base (Python 3.12.4) Completions: conda(base) LSP: Python Line 96, Col 1 UTF-8 CRLF RW Mem 34%

Pré-processamento de dados

- **Balanceamento**
  - Primeiro, precisamos verificar o balanceamento da base:

```
# ==============================================================================
#                      Balanceamento com Undersampling
# ==============================================================================

risk_count = base['Risk'].value_counts()
print('Classe good:', risk_count['good'])
print('Classe bad:', risk_count['bad'])
print('Proportion:', round(risk_count['good'] / risk_count['bad'], 2), ': 1')
risk_count.plot(kind='bar', title='Count (target)',color = ['#1F77B4', '#FF7F0E']);

from imblearn.under_sampling import RandomUnderSampler
import pandas as pd

# Aplicando Undersampling
undersample = RandomUnderSampler(random_state=0)
previsores, classe = undersample.fit_resample(previsores, classe)

# Exibindo as novas distribuições das classes
print(classe.value_counts())
```

Pré-processamento de dados

- **Balanceamento**
  - Primeiro, precisamos verificar o balanceamento da base:

```
# ==============================================================================
#                      Balanceamento com Undersampling
# ==============================================================================

risk_count = base['Risk'].value_counts()
print('Classe good:', risk_count['good'])
print('Classe bad:', risk_count['bad'])
print('Proportion:', round(risk_count['good'] / risk_count['bad'], 2), ': 1')
risk_count.plot(kind='bar', title='Count (target)',color = ['#1F77B4', '#FF7F0E']);

from imblearn.under_sampling import RandomUnderSampler
import pandas as pd

# Aplicando Undersampling
undersample = RandomUnderSampler(random_state=0)
previsores, classe = undersample.fit_resample(previsores, classe)

# Exibindo as novas distribuições das classes
print(classe.value_counts())
```

```
Classe good: 700
Classe bad: 300
Proportion: 2.33 : 1
```

Desbalanceada!
Precisamos aplicar uma das 3 técnicas!

Pré-processamento de dados

- **Balanceamento**
  - Agora, vamos balancear:

```
# ==============================================================================
#                      Balanceamento com Undersampling
# ==============================================================================

risk_count = base['Risk'].value_counts()
print('Classe good:', risk_count['good'])
print('Classe bad:', risk_count['bad'])
print('Proportion:', round(risk_count['good'] / risk_count['bad'], 2), ': 1')
risk_count.plot(kind='bar', title='Count (target)',color = ['#1F77B4', '#FF7F0E']);
```

```
from imblearn.under_sampling import RandomUnderSampler
import pandas as pd

# Aplicando Undersampling
undersample = RandomUnderSampler(random_state=0)
previsores, classe = undersample.fit_resample(previsores, classe)

# Exibindo as novas distribuições das classes
print(classe.value_counts())
```

Pré-processamento de dados

- **Balanceamento**
  - Agora, vamos balancear:

```python
# ==============================================================================
#                      Balanceamento com Undersampling
# ==============================================================================

risk_count = base['Risk'].value_counts()
print('Classe good:', risk_count['good'])
print('Classe bad:', risk_count['bad'])
print('Proportion:', round(risk_count['good'] / risk_count['bad'], 2), ': 1')
risk_count.plot(kind='bar', title='Count (target)',color = ['#1F77B4', '#FF7F0E']);
```

```python
from imblearn.under_sampling import RandomUnderSampler
import pandas as pd

# Aplicando Undersampling
undersample = RandomUnderSampler(random_state=0)
previsores, classe = undersample.fit_resample(previsores, classe)

# Exibindo as novas distribuições das classes
print(classe.value_counts())
```

```
Risk
0        300
1        300
Name: count, dtype: int64
```

Pré-processamento de dados

- **Separação em base de treinamento e testes**
  - Como vimos anteriormente, o aprendizado **supervisionado** é baseado na comparação de um resultado **esperado** com um **obtido**.
  - Para avaliarmos o **percentual de acerto** de um algoritmo, é necessário apresentar para o sistema treinado uma base de dados nunca antes vista.
  - Por esta razão, utilizamos apenas uma **parte** dos dados disponíveis para a fase de treinamento, enquanto o restante é usado para avaliação do método.

Pré-processamento de dados

- **Separação em base de treinamento e testes**

TREINAMENTO

História do crédito	Dívida	Garantias	Renda anual	Risco
Ruim	Alta	Nenhuma	< 15.000	Alto
Desconhecida	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto
Desconhecida	Baixa	Nenhuma	>= 15.000 a <= 35.000	Moderado
Desconhecida	Baixa	Nenhuma	< 15.000	Alto
Desconhecida	Baixa	Nenhuma	> 35.000	Baixo
Desconhecida	Baixa	Adequada	> 35.000	Baixo
Ruim	Baixa	Nenhuma	< 15.000	Alto
Ruim	Baixa	Adequada	> 35.000	Moderado
Boa	Baixa	Nenhuma	> 35.000	Baixo
Boa	Alta	Adequada	> 35.000	Baixo
Boa	Alta	Nenhuma	< 15.000	Alto
Boa	Alta	Nenhuma	>= 15.000 a <= 35.000	Moderado
Boa	Alta	Nenhuma	> 35.0000	Baixo
Ruim	Alta	Nenhuma	>= 15.000 a <= 35.000	Alto

TESTE

História do crédito	Dívida	Garantias	Renda anual	
Ruim	Alta	Adequada	< 15.000	Alto
Desconhecida	Alta	Adequada	< 15.000	Moderado
Desconhecida	Baixa	Nenhuma	> 35.000	Baixo
Boa	Alta	Adequada	>= 15.000 a <= 35.000	Alto

Pré-processamento de dados

- **Separação em base de treinamento e testes**
  - No código, basta executar os seguintes comandos para separar a base em **duas fatias** aleatórias:

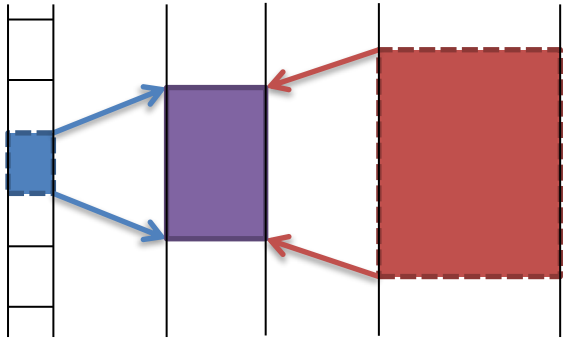
```
# Separando em base de testes e treinamento (usando 25% para teste)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, classe_treinamento, classe_teste =
                train_test_split(previsores, classe, test_size=0.25, random_state=0)
```

  - Com isso, já temos a **base preparada** para a aplicação dos métodos de Aprendizagem de Máquina.
  - **Obs**: é importante lembrar que todas as técnicas de pré-processamento mostradas podem trazer **vantagens** e **desvantagens** aos resultados. É necessário **testar** o efeito de cada uma!

Pré-processamento de dados

- **Ajuste de escalas**
  - Diferentes atributos numéricos podem possuir **escalas diferentes**.
  - Se a diferença for muito grande entre as escalas, alguns algoritmos poderão sofrer severos **prejuízos**.
  - Métodos que utilizam o **cálculo de distância** irão dar muito mais importância para valores que estão em escalas maiores.

  - Exemplos de dados com escalas diferentes
    - Idade e salário
    - Número de acessos e avaliação média



Pré-processamento de dados

- **Ajuste de escalas**
  - Na base em que estamos trabalhando já existem, originalmente, 3 variáveis numéricas com escalas distintas:

➢ Idade – **Numérica discreta**

➢ Valor do crédito solicitado **– Numérica contínua**

➢ Meses de duração do empréstimo – **Numérica discreta**

  - Enquanto a **idade** corresponde a um valor apropriado para uma pessoa adulta, o **valor do crédito** pode chegar a milhares.
  - Enquanto isso, o **número de meses** pode ser bem pequeno.

Pré-processamento de dados

- **Ajuste de escalas**
  - Existem duas fórmulas básicas para ajuste de escalas de variáveis:

1. **Padronização** (Standardization)

$$\hat{x} = \frac{x_i - média(x)}{desvio\ padrão(x)}$$

2. **Normalização** (Normalization)

$$\hat{x} = \frac{x_i - mínimo(x)}{máximo(x) - \ mínimo(x)}$$

- **Separação de variáveis**
  - No código, vamos utilizar a classe StandardScaler para padronizar o valor de todos os atributos.

```
# ==========================================================================
#                    Padronização dos dados
# ==========================================================================

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
previsores_treinamento = scaler.fit_transform(previsores_treinamento)
previsores_teste = scaler.transform(previsores_teste)
```

Todos os dados (agora numéricos) serão deixados na mesma escala.

Pré-processamento de dados

- **Bases de dados**
  - Existem diversas bases de dados **públicas** disponíveis na internet.
  - Esses dados podem ser usados para pesquisas científicas de interesse público ou pessoal.
  - Dois repositórios de **bases de dados públicas** muito usados são:

https://archive.ics.uci.edu/datasets

UCI Machine Learning Repository
Center for Machine Learning and Intelligent Systems

https://www.kaggle.com/datasets

kaggle™

Pré-processamento de dados

- **Bases de dados**
  - Além disso, a política de **abertura de dados** está se expandindo.
  - Governos de países (inclusive o Brasil) estão disponibilizando publicamente dados que podem ser usados para análises e pesquisas científicas.
  - Com isso, a comunidade científica pode auxiliar na identificação de problemas e no desenvolvimento de planos de ação social.
  - Exemplo:

http://dados.gov.br/

dados.gov.br

PORTAL BRASILEIRO DE DADOS ABERTOS

Pré-processamento de dados

- **Tarefa**

➢ Encontre uma base de dados que corresponda a um problema de **classificação** que seja de seu interesse.

➢ Os dados devem estar em um **formato similar** à base usada como exemplo em sala.

➢ Faça uma análise da base escolhida e identifique os **tipos** de suas variáveis e as **inconsistências** presentes.

➢ Aplique as técnicas de **pré-processamento** de dados para preparar a base.

FIM